

**Министерство науки и высшего образования
Российской Федерации ФЕДЕРАЛЬНОЕ
ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

**«АНАЛИЗ ДАННЫХ. ПОСТРОЕНИЕ ИНФОЛОГИЧЕСКОЙ МОДЕЛИ
ДАННЫХ БД»**

по дисциплине «Проектирование и реализация баз данных»

Обучающийся: Голинский Данила Владимирович

Факультет прикладной информатики

Группа: К3240

Направление подготовки 09.03.03 Прикладная
информатика

Образовательная программа: Мобильные и сетевые
технологии

Преподаватель: Говорова Марина Михайловна

Санкт-
Петербург
2026/2027

Тема: Запросы на выборку и модификацию данных. Представления. Работа с индексами. Вариант: 17 (Телефонный провайдер)

1. Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

2. Практическое задание

- Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию).
- Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
- Изучить графическое представление запросов и просмотреть историю запросов.
- Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

3. Схема базы данных



4. Выполнение

Перед выполнением запросов был осуществлен переход к рабочей схеме:

```
SET search_path TO provider_schema;
```

Задание 2. Запросы к базе данных

2.1. Вывести суммарное время переговоров каждого абонента за заданный период (март 2025 г.).

```
SELECT
  s.full_name AS "ФИО абонента",
  SUM(c.duration_minutes) AS "Суммарное время (мин)"
FROM subscribers s
JOIN calls c ON s.subscriber_id = c.subscriber_id
WHERE c.call_date BETWEEN '2025-03-01' AND '2025-03-31'
GROUP BY s.full_name;
```

Data Output Messages Notifications

	ФИО абонента character varying (100)	Суммарное время (мин) numeric
1	Голинский Данила Владимирович	10.50
2	Иванов Иван Иванович	5.00

2.2. Найти среднюю продолжительность разговора абонента АТС.

```
SELECT
  s.full_name AS "ФИО абонента",
  ROUND(AVG(c.duration_minutes), 2) AS "Средняя продолжительность (мин)"
FROM subscribers s
JOIN calls c ON s.subscriber_id = c.subscriber_id
GROUP BY s.full_name;
```

Data Output Messages Notifications		
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈		
	ФИО абонента character varying (100)	Средняя продолжительность (мин) numeric
1	Иванов Иван Иванович	5.00
2	Голинский Данила Владимирович	10.50

2.3. Вывести количество междугородных переговоров каждого абонента.

```

SELECT
    s.full_name AS "ФИО абонента",
    COUNT(c.call_id) AS "Кол-во междугородных звонков"
FROM subscribers s
JOIN calls c ON s.subscriber_id = c.subscriber_id
JOIN tariff_zones tz ON c.zone_code = tz.zone_code
WHERE tz.zone_name != s.city
GROUP BY s.full_name;

```

Data Output Messages Notifications		
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈		
	ФИО абонента character varying (100)	Кол-во междугородных звонков bigint
1	Иванов Иван Иванович	1

2.4. Вывести список абонентов, продолжительность разговоров которых за прошедшую неделю была максимальной.

```

WITH RankedDurations AS (
    SELECT
        subscriber_id,
        SUM(duration_minutes) as sum_dur,
        RANK() OVER (ORDER BY SUM(duration_minutes) DESC) as rank_pos
    FROM calls
    WHERE call_date BETWEEN '2025-03-01' AND '2025-03-07'
    GROUP BY subscriber_id
)
SELECT s.full_name AS "ФИО абонента", rd.sum_dur AS "Максимальная
продолжительность"

```

```
FROM subscribers s
JOIN RankedDurations rd ON s.subscriber_id = rd.subscriber_id
WHERE rd.rank_pos = 1;
```

Data Output Messages Notifications		
	ФИО абонента character varying (100)	Максимальная продолжительность numeric
1	Голинский Данила Владимирович	10.50

2.5. Сколько звонков было сделано в каждый из городов России.

```
SELECT
    tz.zone_name AS "Город (Зона)",
    COUNT(c.call_id) AS "Количество звонков"
FROM calls c
JOIN tariff_zones tz ON c.zone_code = tz.zone_code
WHERE tz.country = 'Россия'
GROUP BY tz.zone_name;
```

Data Output Messages Notifications		
	Город (Зона) character varying (50)	Количество звонков bigint
1	Санкт-Петербург	1

2.6. Вывести список абонентов, звонивших только в ночное время (с 00:00 до 06:00).

```
SELECT s.full_name AS "ФИО абонента"
FROM subscribers s
JOIN calls c ON s.subscriber_id = c.subscriber_id
GROUP BY s.subscriber_id, s.full_name
HAVING COUNT(*) = SUM(CASE WHEN c.call_time >= '00:00:00' AND c.call_time <
'06:00:00' THEN 1 ELSE 0 END);
```

Data Output	Messages	Notifications
		
		
		
ФИО абонента character varying (100) 		

2.7. Вывести список абонентов, время разговоров которых превышает среднее для этой же зоны.

```

SELECT DISTINCT
  s.full_name AS "ФИО абонента",
  c.duration_minutes AS "Длительность звонка",
  tz.zone_name AS "Зона звонка"
FROM subscribers s
JOIN calls c ON s.subscriber_id = c.subscriber_id
JOIN tariff_zones tz ON c.zone_code = tz.zone_code
WHERE c.duration_minutes > (
  SELECT AVG(duration_minutes)
  FROM calls c2
  WHERE c2.zone_code = c.zone_code
);

```

Data Output	Messages	Notifications
		
		
		
ФИО абонента character varying (100) 	Длительность звонка numeric (8,2) 	Зона звонка character varying (50) 

Задание 3. Создать представления

3.1. Представление: Содержащее сведения обо всех абонентах и их переговорах за прошедший месяц.

```

CREATE VIEW vw_subscribers_last_month_calls AS
SELECT
  s.full_name AS subscriber_name,
  s.phone_number,
  c.dialed_number,
  c.call_date,
  c.duration_minutes,

```

```

c.cost
FROM subscribers s
JOIN calls c ON s.subscriber_id = c.subscriber_id
WHERE c.call_date >= '2025-03-01' AND c.call_date <= '2025-03-31';

SELECT * FROM vw_subscribers_last_month_calls;

```

	subscriber_name character varying (100)	phone_number character varying (20)	dialled_number character varying (20)	call_date date	duration_minutes numeric (8,2)	cost numeric (8,2)
1	Голинский Данила Владимирович	+79111234567	+79001112233	2025-03-01	10.50	0.00
2	Иванов Иван Иванович	+79217654321	+375291112233	2025-03-02	5.00	75.00

3.2. Представление: Найти самую популярную зону звонков за истекший год.

```

CREATE OR REPLACE VIEW vw_popular_zone_last_year AS
WITH RankedZones AS (
    SELECT
        tz.zone_name,
        COUNT(c.call_id) AS total_calls,
        RANK() OVER (ORDER BY COUNT(c.call_id) DESC) as rank_pos
    FROM calls c
    JOIN tariff_zones tz ON c.zone_code = tz.zone_code
    WHERE c.call_date >= '2025-01-01' AND c.call_date <= '2025-12-31'
    GROUP BY tz.zone_name
)
SELECT zone_name, total_calls
FROM RankedZones
WHERE rank_pos = 1;

SELECT * FROM vw_popular_zone_last_year;

```

	zone_name character varying (50)	total_calls bigint
1	Беларусь	1
2	Санкт-Петербург	1

Запросы на модификацию данных с использованием подзапросов

1. Операция UPDATE (Обновление)

Формулировка: Начислить бонус в размере 100 рублей на баланс всем абонентам, которые подключены к тарифу «Безлимит 2026».

```
UPDATE subscribers
SET current_balance = current_balance + 100
WHERE subscriber_id IN (
    SELECT st.subscriber_id
    FROM subscriber_tariffs st
    JOIN tariff_plans tp ON st.tariff_id = tp.tariff_id
    WHERE tp.tariff_name = 'Безлимит 2026'
);
```

Состояние ДО (скрипт для проверки):

```
SELECT s.full_name, s.current_balance
FROM subscribers s
JOIN subscriber_tariffs st ON s.subscriber_id = st.subscriber_id
JOIN tariff_plans tp ON st.tariff_id = tp.tariff_id
WHERE tp.tariff_name = 'Безлимит 2026';
```

Data Output Messages Notifications		
		
	full_name character varying (100)	current_balance numeric (10,2)
1	Голинский Данила Владимирович	450.00

Состояние ПОСЛЕ:

 Provider/postgres@PostgreSQL 16

 No limit 

Query

Query History

```
1 UPDATE subscribers
2 SET current_balance = current_balance + 100
3 WHERE subscriber_id IN (
4     SELECT st.subscriber_id
5     FROM subscriber_tariffs st
6     JOIN tariff_plans tp ON st.tariff_id = tp.tari
7     WHERE tp.tariff_name = 'Безлимит 2026'
8 );
9
```

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 44 msec.

Data Output			Messages	Notifications
        				
	full_name character varying (100)		current_balance numeric (10,2)	
1	Голинский Данила Владимирович		550.00	

2. Операция INSERT (Добавление)

Формулировка: Добавить новый платеж на 500 рублей для абонента по его номеру телефона.

```
INSERT INTO payments (subscriber_id, amount, payment_date, due_date, payment_status)
VALUES (
  (SELECT subscriber_id FROM subscribers WHERE phone_number = '+79111234567'),
  500.00,
  CURRENT_DATE,
  CURRENT_DATE,
  'Оплачен'
);
```

Состояние ДО (скрипт для проверки):

```
SELECT * FROM payments ORDER BY payment_date DESC;
```

Data Output Messages Notifications							
	payment_id [PK] integer	subscriber_id integer	amount numeric (10,2)	payment_date date	due_date date	payment_status public.payment_status_enum	
1	2	2	350.00	2025-02-15	2025-02-15	Оплачен	
2	1	1	1200.00	2025-01-10	2025-01-10	Оплачен	

Состояние ПОСЛЕ:

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 46 msec.

Data Output Messages Notifications							
	payment_id [PK] integer	subscriber_id integer	amount numeric (10,2)	payment_date date	due_date date	payment_status public.payment_status_enum	
1	3	1	500.00	2026-05-10	2026-05-10	Оплачен	
2	2	2	350.00	2025-02-15	2025-02-15	Оплачен	
3	1	1	1200.00	2025-01-10	2025-01-10	Оплачен	

3. Операция DELETE (Удаление)

Формулировка: Удалить историю звонков абонентов, проживающих в городе 'Москва'. Можно транзакцию добавить begin; - чтобы потом вернуть через rollback;

```
DELETE FROM calls
WHERE subscriber_id IN (
    SELECT subscriber_id FROM subscribers WHERE city = 'Москва'
);
```

Состояние ДО (скрипт для проверки):

```
SELECT c.*, s.city
FROM calls c
JOIN subscribers s ON c.subscriber_id = s.subscriber_id
WHERE s.city = 'Москва';
```

call_id	subscriber_id	dial_number	call_date	call_time	duration	zone_code	tariff_id	cost	city
bigint	integer	character varying (20)	date	time without time zone	numeric (8,2)	character varying (10)	integer	numeric (8,2)	character varying (50)
1	2	+375291112233	2025-03-02	15:30:00	5.00	INT-BY	1	75.00	Москва

Состояние ПОСЛЕ (скрипт для проверки):

```
SELECT c.*, s.city
FROM calls c
JOIN subscribers s ON c.subscriber_id = s.subscriber_id
WHERE s.city = 'Москва';
```

Data Output Messages Notifications										
call_id	subscriber_id	dialled_number	call_date	call_time	duration_minutes	zone_code	tariff_id_at_call	cost	city	
bigint	integer	character varying (20)	date	time without time zone	numeric (8,2)	character varying (10)	integer	numeric (8,2)	character varying (50)	

Создание индексов и планы запросов (EXPLAIN)

Для корректной демонстрации работы индексов на малом объеме данных отключаем последовательное сканирование:

```
SET enable_seqscan = OFF;
```

Анализ 1: Простой индекс (по одному полю)

Запрос БЕЗ индекса:

```
EXPLAIN ANALYZE SELECT * FROM calls WHERE dialed_number = '+79001112233';
```

Data Output		Messages	Notifications
			
	QUERY PLAN		
	text		
1	Seq Scan on calls (cost=10000000000.00..10000000015.62 rows=2 width=152) (actual time=0.018..0.019 rows=1 loops...		
2	Filter: ((dialed_number)::text = '+79001112233'::text)		
3	Rows Removed by Filter: 2		
4	Planning Time: 0.087 ms		
5	Execution Time: 0.033 ms		

Создание индекса и запрос С индексом:

```
CREATE INDEX idx_dialed_number ON calls(dialed_number);  
EXPLAIN ANALYZE SELECT * FROM calls WHERE dialed_number = '+79001112233';
```

Data Output		Messages	Notifications
	QUERY PLAN		
	text		
1	Index Scan using idx_dialed_number on calls (cost=0.13..8.15 rows=1 width=152) (actual time=0.058..0.059 rows=1 loop...		
2	Index Cond: ((dialed_number)::text = '+79001112233'::text)		
3	Planning Time: 0.790 ms		
4	Execution Time: 0.077 ms		

Анализ 2: Составной индекс (по двум полям)

Запрос БЕЗ индекса:

```
EXPLAIN ANALYZE SELECT * FROM subscribers WHERE city = 'Санкт-Петербург' AND  
country = 'Россия';
```

Data Output		Messages	Notifications
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈			
	QUERY PLAN text 🔒		
1	Seq Scan on subscribers (cost=10000000000.00..10000000011.20 rows=1 width=962) (actual time=0.026..0.027 rows=1 loop...		
2	Filter: (((city)::text = 'Санкт-Петербург'::text) AND ((country)::text = 'Россия'::text))		
3	Rows Removed by Filter: 1		
4	Planning Time: 0.134 ms		
5	Execution Time: 0.047 ms		

Создание составного индекса и запрос С индексом:

```
CREATE INDEX idx_city_country ON subscribers(city, country);
EXPLAIN ANALYZE SELECT * FROM subscribers WHERE city = 'Санкт-Петербург' AND
country = 'Россия';
```

Data Output		Messages	Notifications
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈			
	QUERY PLAN text 🔒		
1	Index Scan using idx_city_country on subscribers (cost=0.13..8.15 rows=1 width=962) (actual time=0.078..0.079 rows=1 loop...		
2	Index Cond: (((city)::text = 'Санкт-Петербург'::text) AND ((country)::text = 'Россия'::text))		
3	Planning Time: 0.824 ms		
4	Execution Time: 0.096 ms		

Удаление созданных индексов и возврат настроек:

```
DROP INDEX idx_dialed_number;
DROP INDEX idx_city_country;
SET enable_seqscan = ON;
```

5. Выводы

В ходе выполнения лабораторной работы были успешно получены практические навыки написания DML-запросов (Data Manipulation Language) к СУБД PostgreSQL. Были решены следующие задачи:

- Сформированы сложные выборки данных с использованием многотабличных соединений (JOIN), агрегатных функций (SUM, AVG, COUNT) и группировок (GROUP BY).
- Разработаны коррелированные подзапросы и обобщенные табличные выражения (CTE) для решения аналитических задач по абонентам.
- Созданы представления (VIEW), инкапсулирующие сложную бизнес-логику для упрощения доступа к агрегированным данным.
- Изучены методы модификации данных (INSERT, UPDATE, DELETE) с динамическим вычислением значений через вложенные подзапросы.
- Освоен инструмент профилирования запросов EXPLAIN ANALYZE. Практически доказана эффективность применения простых и составных B-Tree индексов: проведенный анализ планов выполнения показал изменение стратегии с полного сканирования таблицы (Seq Scan) на сканирование по индексу (Index Scan).